

Design and Analysis of Algorithms

Series Summation and Recurrence Relations

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Outline

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Mathematics of Algorithm Complexity

An algorithm typically consists of loop and iteration structure

- complexity $\leadsto$ series summation

Methods of calculating series summation

- general term formula $\leadsto$ exact result
 - estimate the upper bound of summation $\leadsto$ approximate result
-

Algorithm may consist of recursive structure

- complexity $\leadsto$ recurrence relations

Methods for solving recurrence relations

- recurrence relation is simple: direct iteration + substitution iteration
- recurrence relation is complex: simplification + recursion tree
- general case: master theorem

Concepts of Sequences and Series

Sequence: an ordered list of numbers; the numbers in this ordered list are called the “terms” of the sequence.

Concepts of Sequences and Series

Sequence: an ordered list of numbers; the numbers in this ordered list are called the “terms” of the sequence.

Series: the sum of all the terms of a sequence; the resulting value, are called the “sum” or the “summation”.

Concepts of Sequences and Series

Sequence: an ordered list of numbers; the numbers in this ordered list are called the “terms” of the sequence.

Series: the sum of all the terms of a sequence; the resulting value, are called the “sum” or the “summation”.

Example. 1, 2, 3, 4 is a sequence, with terms “1”, “2”, “3”, “4”; the corresponding series is the sum “ $1 + 2 + 3 + 4$ ”, and the value of the series is 10.

We first recall three classical sequences.

Arithmetic Sequence

Arithmetic Sequence

$$a, a + d, \dots, a + (n - 1)d$$

$$a_i = a + (i - 1)d$$

$$\text{common difference} = d \neq 0$$

Arithmetic Series

$$S(n) = \sum_{i=1}^n a_i = \frac{n(a_1 + a_n)}{2} = \frac{n(2a + (n - 1)d)}{2}$$

Geometric Sequence

Geometric Sequence

$$a, ar, \dots, ar^{n-1}$$

$$a_i = ar^{i-1}$$

$$\text{common ratio} = r \neq 1$$

Geometric Series

$$S(n) = \sum_{i=1}^n ar^{i-1} = a + ar + ar^2 + \dots + ar^{n-1}$$

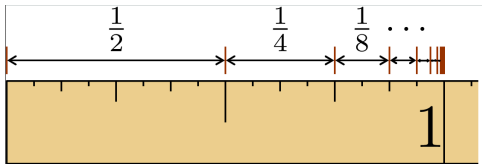
$$rS(n) = \sum_{i=1}^n ar^i = ar + ar^2 + ar^3 + \dots + ar^n$$

$$\Rightarrow S(n) - rS(n) = a - ar^n \Rightarrow$$

$$S(n) = a \left(\frac{1 - r^n}{1 - r} \right) \quad \lim_{n \rightarrow \infty} S(n) = \frac{a}{1 - r}, |r| < 1$$

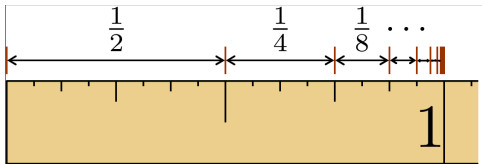
Visualization of Geometric Series

$$S(n) = \frac{1}{2} + \frac{1}{4} + \dots$$

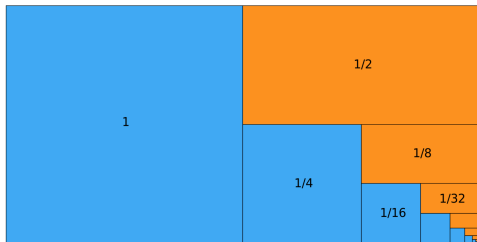


Visualization of Geometric Series

$$S(n) = \frac{1}{2} + \frac{1}{4} + \dots$$



$$S(n) = 1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots$$



Applications of Geometric Sequence

Geometric Series is among the simplest examples of infinite series with finite sums (although not all of them have this property).

Geometric series are used throughout mathematics, have important applications in physics, engineering, biology, economics, computer science, queueing theory, and finance.

- Repeating decimals (e.g., $0.77777\ldots$) are rational
- Fractal geometry
- Zeno's paradoxes
- Economics: the present value of an annuity
- number of total Bitcoins $\leq 2.1 \times 10^8$

Harmonic Sequence

Harmonic Sequence (whose inverse forms an arithmetic sequence)

$$a_i = \frac{1}{i} : 1, \frac{1}{2}, \dots, \frac{1}{n}$$

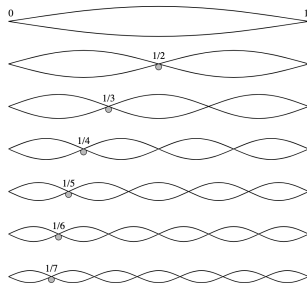
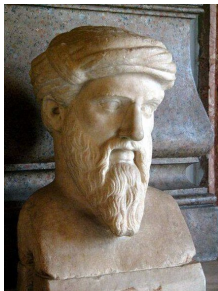
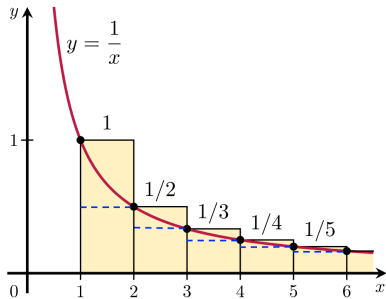


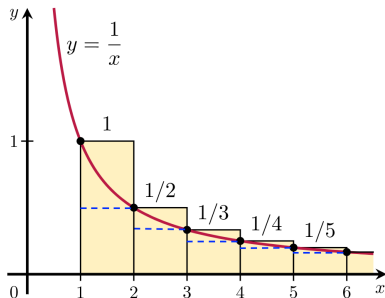
Figure: Pythagoras

Calculation of Harmonic Series: Integral Test



$$S(n) = \Theta(\ln n)$$

Calculation of Harmonic Series: Integral Test



$$S(n) = \Theta(\ln n)$$

Lower bound:
$$S(n) = \sum_{i=1}^n \frac{1}{i} > \int_{i=1}^{n+1} \frac{1}{x} dx = \ln(n+1)$$

Upper bound:
$$S(n) = \sum_{i=1}^n \frac{1}{i} = 1 + \left(\frac{1}{2} + \cdots + \frac{1}{n} \right) < 1 + \int_{i=1}^n \frac{1}{x} dx = \ln n + 1$$

Interesting Properties of These Sequences

The middle term is the “mean” of its two neighbors

Interesting Properties of These Sequences

The middle term is the “mean” of its two neighbors

- arithmetic sequences

$$\text{arithmetic mean: } a_{i+1} = \frac{a_i + a_{i+2}}{2}$$

Interesting Properties of These Sequences

The middle term is the “mean” of its two neighbors

- arithmetic sequences

$$\text{arithmetic mean: } a_{i+1} = \frac{a_i + a_{i+2}}{2}$$

- geometric sequences

$$\text{geometric mean: } a_{i+1} = \sqrt{a_i \cdot a_{i+2}}$$

Interesting Properties of These Sequences

The middle term is the “mean” of its two neighbors

- arithmetic sequences

$$\text{arithmetic mean: } a_{i+1} = \frac{a_i + a_{i+2}}{2}$$

- geometric sequences

$$\text{geometric mean: } a_{i+1} = \sqrt{a_i \cdot a_{i+2}}$$

- harmonic sequences

$$\text{harmonic mean: } a_{i+1} = \frac{2}{\frac{1}{a_i} + \frac{1}{a_{i+2}}}$$

Exact Series Summation

$$\begin{aligned}\sum_{i=1}^n i2^{i-1} &= \sum_{i=1}^n i(2^i - 2^{i-1}) && //\text{split terms} \\&= \sum_{i=1}^n i2^i - \sum_{i=1}^n i2^{i-1} \\&= \sum_{i=1}^n i2^i - \sum_{i=0}^{n-1} (i+1)2^i && //\text{substitute subscripts} \\&= \sum_{i=1}^n i2^i - \sum_{i=0}^{n-1} i2^i - \boxed{\sum_{i=0}^{n-1} 2^i} && //\text{split terms} \\&= n2^n - (2^n - 1) = (n-1)2^n + 1 && //\text{geometric series}\end{aligned}$$

Approximate Series Summation

Amplification method

① $\sum_{i=1}^n a_i \leq n a_{\max}$ (coarse)

- ② Suppose $\exists 0 < r < 1$ such that $a_{i+1}/a_i \leq r$ holds $\forall k \geq 0$, then the sequence can be bounded by a geometric series.

$$\sum_{i=0}^n a_i \leq \sum_{i=0}^n a_0 r^i = a_0 \frac{1 - r^{n+1}}{1 - r}$$

Example of Amplification Method

Estimate an upper bound for $\sum_{i=1}^n \frac{i}{3^i}$

Solution.

$$a_i = \frac{i}{3^i}, a_{i+1} = \frac{i+1}{3^{i+1}} \Rightarrow$$
$$\frac{a_{i+1}}{a_i} = \frac{1}{3} \frac{i+1}{i} \leq \frac{2}{3}$$

Applying the amplification method, we obtain:

$$\sum_{i=1}^n \frac{i}{3^i} < \sum_{i=1}^{\infty} \frac{1}{3} \left(\frac{2}{3}\right)^{i-1} = \frac{1}{3} \frac{1}{1 - \frac{2}{3}} = 1$$

Binary Search Algorithm

Algorithm 1: BinarySearch(A, l, r, x)

Input: $A[l, r]$, target element x

Output: j

```
1:  $l \leftarrow 1, r \leftarrow n$ ;  
2: while  $l \leq r$  do  
3:    $m \leftarrow \lfloor (l + r)/2 \rfloor$ ;  
4:   if  $A[m] = x$  then return  $m$ ;  $//x$  is the median  
5:   else if  $A[m] > x$  then  $r \leftarrow m - 1$ ;  
6:   else  $l \leftarrow m + 1$ ;  
7: end  
8: return 0
```

Demo of Binary Search

1st compare: $3.5 < 4$

1	2	3	4	5	6	7
			3.5			

2nd compare: $3.5 > 2$

1	2	3	4	5	6	7
	3.5					

3rd compare: $3.5 > 3$

1	2	3	4	5	6	7
		3.5				

Input Size n

Ideal case. $n = 2^k - 1$

Input Size n

Ideal case. $n = 2^k - 1$

Q. Why do we call $n = 2^k - 1$ as ideal case?

Input Size n

Ideal case. $n = 2^k - 1$

Q. Why do we call $n = 2^k - 1$ as ideal case?

A. Because the subproblem size remains of the form $2^i - 1$

Input Size n

Ideal case. $n = 2^k - 1$

Q. Why do we call $n = 2^k - 1$ as ideal case?

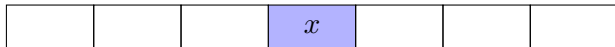
A. Because the subproblem size remains of the form $2^i - 1$

There are $2n + 1$ possible outcomes for x :

- x in the array: n
- x not in the array: fall into $n + 1$ intervals

Number of Input x Requiring t Comparisons ($n = 7, k = 3$)

$t = 1 : 1$



$t = 2 : 2$



$t = 3 : 4$



- for $t \in [k - 1]$, # possible input elements that requires t times compares is 2^{t-1}
- for $t = k$, # possible input elements that requires k times compares is $2^{k-1} + (n + 1)$

Average-case Complexity of Binary Search

Let $n = 2^k - 1$, assume x appears at each position with the same probability:

$$\begin{aligned} T(n) &= \frac{1}{2n+1} \left(\sum_{t=1}^{k-1} t 2^{t-1} + k(2^{k-1} + n + 1) \right) \\ &= \frac{1}{2n+1} \left(\sum_{t=1}^{k-1} t 2^{t-1} + k(2^{k-1} + 2^k) \right) \\ &= \frac{1}{2n+1} \left((k-2)2^{k-1} + 1 + k2^k + k2^{k-1} \right) // \text{use previous result} \\ &= \frac{k2^k - 2^k + 1 + k2^k}{2n+1} \\ &= \frac{(2k-1)2^k + 1}{2^{k+1} - 1} \approx k - \frac{1}{2} = \Theta(\log n) \end{aligned}$$

Outline

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Motivation

Recursion and Iteration are two commonly used programming paradigms

Common: A solution to a problem is obtained by combining solutions to subproblems of smaller size.

In this case, time complexity functions can be expressed as recurrence relations.

How do we solve recurrence relations?

Definition 1 (Recurrence Relation)

Let $a_0, a_1, \dots, a_n$ be a sequence, abbreviated as $\{a_n\}$. A recurrence relation defines each term of a sequence using preceding term(s), and always state the initial term of the sequence.

Recurrence relation captures the dependence of a term on its preceding terms.

Solution. Given recurrence relation for a sequence $\{a_n\}$ together with some initial values, compute the general term formula of a_n .

- general term formula: a function of n , without involving other terms

Example of Recurrence Relation: Fibonacci Number

The Fibonacci sequence:

1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...

Recurrence relation: $f_n = f_{n-1} + f_{n-2}$

Initial values: $f_0 = 1, f_1 = 1$



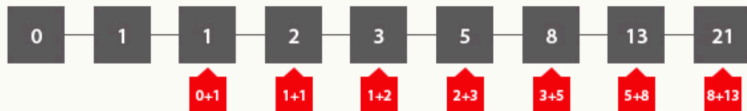
Figure: Fibonacci

$$f_n = \frac{1}{\sqrt{5}} \left(\frac{1 + \sqrt{5}}{2} \right)^{n+1} - \frac{1}{\sqrt{5}} \left(\frac{1 - \sqrt{5}}{2} \right)^{n+1}$$

THE GOLDEN RATIO

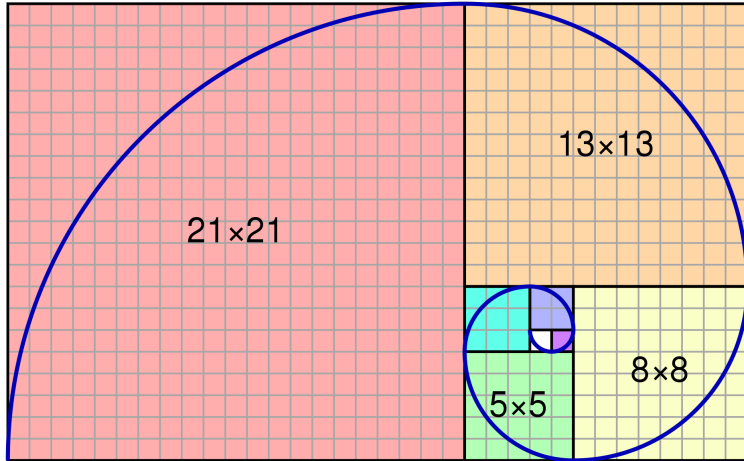
WHAT IS THE GOLDEN RATIO?

The golden ratio originates from a series of numbers called the Fibonacci sequence. Beginning with 0 and 1, each number in the Fibonacci sequence is derived by adding the two previous numbers in the sequence together.



As the numbers in the sequence get larger and larger, the ratio between them gets closer and closer to **1:1.618**. That's the golden ratio.

Visualizing the Golden Ratio



The Golden Ratio is Everywhere (Plant)

— PLANTS —



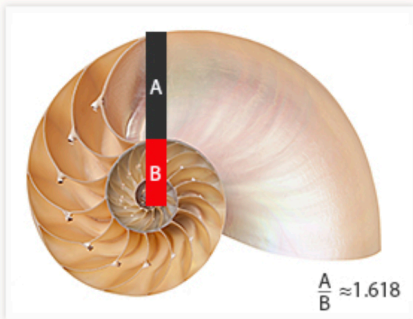
Sunflower



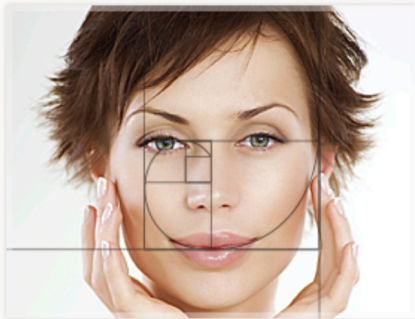
Spiral Aloe

The Golden Ratio is Everywhere (Animal)

— ANIMALS —



Nautilus Shell



Human Face

The Golden Ratio is Everywhere (Animal)

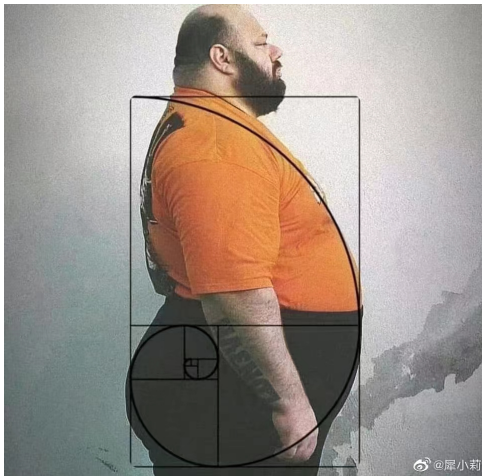
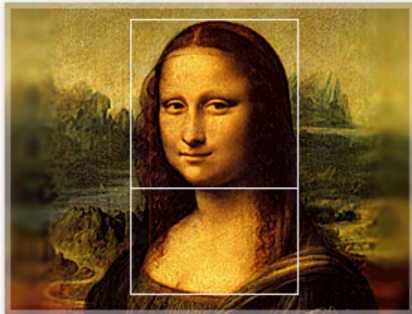


Figure: 男人四十一枝花

The Golden Ratio is Everywhere (Art)

— ART —



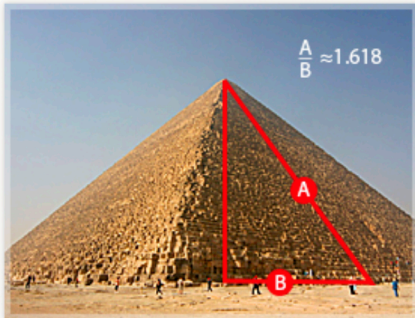
Da Vinci's Mona Lisa



Dali's Sacrament of the Last Supper

The Golden Ratio is Everywhere (Architecture)

— ARCHITECTURE —



Great Pyramid of Giza



Parthenon

The Golden Ratio is Everywhere (Typography)

— TYPOGRAPHY —

Use headline and body text sizes that are the golden ratio to one another. For example, a 20 pt headline would call for roughly 12 pt body text.

LOREM IPSUM DOLOR!

Nam ac tincidunt eros. Phasellus maximus dolor
quis ante congue pharetra. Suspendisse potenti.
Aliquam fringilla ultricies dapibus. Morbi id lacus
ac mauris porta tempus nec in nibh.

Suspendisse nulla libero, elementum eget quam
vulputate, varius commodo magna. Ut mollis
viverra quam, ut accumsan lacus consequat in.
Duis aliquam ullamcorper ante ac convallis.
Nulla at nulla in urna facilisis porttitor.

} 20 pt

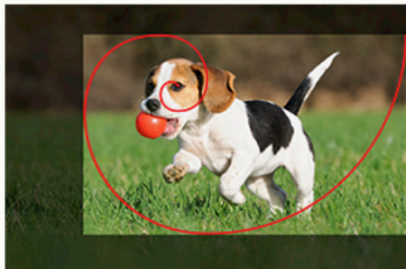
} 12 pt

$$\frac{20}{12} \approx 1.6$$

The Golden Ratio is Everywhere (Sizing/Cropping Images)

— SIZING/CROPPING IMAGES —

Use the golden ratio as your guide for image proportions and for drawing focus to the most important elements.



The Golden Ratio is Everywhere (Shapes and Symbols)

— SHAPES AND SYMBOLS —

Use the golden ratio to add interest to vector-based shapes. Many major companies, for example, have used golden proportions in the design of their logos.



$$\frac{A}{B} \approx 1.618$$



Next, we introduce three methods for solving recurrence relation.

Outline

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Steps of Direct Iteration

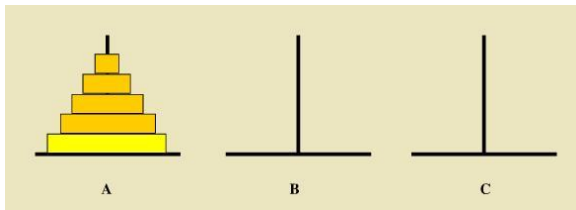
When the recurrence relation is simple, i.e., $F(n)$ only relies on $F(n - 1)$, we use direct iteration.

- ➊ Continuously substitute the “right-hand” side of formula with the “right-right-hand”
- ➋ After each substitution, a new term emerges in the series as n decreases
- ➌ Stop substituting until reaching the initial values
- ➍ Calculate the series with the initial values
- ➎ Use mathematical induction to check the correctness of the solution

Remark. Mathematical induction is useful for checking whether the proposed solution is correct. When the correctness is evident, it is not necessary.

Example: Tower of Hanoi Problem

Origin. *When creating the world, Brahma also built three diamond rods and 64 golden disks of different sizes. At the beginning, the disks are placed in ascending order of size on one rod, the smallest at the top, thus making a conical shape. Brahmin priests have been moving these disks from one rod to another in accordance with the immutable rules of Brahma since that time. When the last move is completed, the world will end.*



Problem as a Puzzle

Problem Abstract. There are three rods (labeled as A, B, C) with n of disks of different sizes. At the beginning, the disks are arranged in a neat stack in ascending order of size on rod A . The objective is to figure out the minimum number of moves $T(n)$ required to move the entire stack to rod C , subject to the following rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
- No larger disk may be placed on top of a smaller disk.

Problem as a Puzzle

Problem Abstract. There are three rods (labeled as A, B, C) with n of disks of different sizes. At the beginning, the disks are arranged in a neat stack in ascending order of size on rod A . The objective is to figure out the minimum number of moves $T(n)$ required to move the entire stack to rod C , subject to the following rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
- No larger disk may be placed on top of a smaller disk.

Example. $n = 1, T(1) = 1$; $n = 2, T(2) = 3$; $n = 3, T(3) = 7$;

Problem as a Puzzle

Problem Abstract. There are three rods (labeled as A, B, C) with n of disks of different sizes. At the beginning, the disks are arranged in a neat stack in ascending order of size on rod A . The objective of figure out the minimum number of moves $T(n)$ required to move the entire stack to rod C , subject to the following rules:

- Only one disk can be moved at a time.
- Each move consists of taking the upper disk from one of the stacks and placing it on top of another stack or on an empty rod.
- No larger disk may be placed on top of a smaller disk.

Example. $n = 1, T(1) = 1$; $n = 2, T(2) = 3$; $n = 3, T(3) = 7$;

general form $T(n) = ?$

Recursive Algorithm for the Tower of Hanoi

Algorithm 2: Hanoi(A, C, n) // move n disks from A to C

Input: $A(n), B(0), C(0)$

Output: $A(0), B(0), C(n)$

```
1: if  $n = 1$  then move ( $A, C$ ); //move one disk from  $A$  to  $C$ 
2: else
3:   Hanoi( $A, B, n - 1$ ) // use  $C$  as swap tower;
4:   move ( $A, C$ );
5:   Hanoi( $B, C, n - 1$ ) // use  $A$  as swap tower;
6: end
```

Let $T(n)$ be the number of moves required to move n disks

- $T(n) = 2T(n - 1) + 1$
- $T(1) = 1$

Complexity Analysis: Direct Iteration

$$\left. \begin{array}{l} T(n) = 2T(n-1) + 1 \\ T(1) = 1 \end{array} \right\} \Rightarrow T(n) = 2^n - 1$$

Complexity Analysis: Direct Iteration

$$\left. \begin{array}{l} T(n) = 2T(n-1) + 1 \\ T(1) = 1 \end{array} \right\} \Rightarrow T(n) = 2^n - 1$$

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2(2T(n-2) + 1) + 1 \\ &= 2^2T(n-2) + 2 + 1 \\ &= \dots \\ &= 2^{n-1}T(1) + \underline{2^{n-2} + \dots + 2 + 1} // \text{reaching the initial terms} \\ &= 2^{n-1} \cdot 1 + 2^{n-1} - 1 // \text{substitute with initial values} \\ &= 2^n - 1 \end{aligned}$$

More about Hanoi Tower

Is there a better algorithm?

More about Hanoi Tower

Is there a better algorithm?

No! Tower of Hanoi is an intractable problem, no polynomial-time algorithm is known.

More about Hanoi Tower

Is there a better algorithm?

No! Tower of Hanoi is an intractable problem, no polynomial-time algorithm is known.

Q. 1 move/s, how long would it take to move 64 disks?

More about Hanoi Tower

Is there a better algorithm?

No! Tower of Hanoi is an intractable problem, no polynomial-time algorithm is known.

Q. 1 move/s, how long would it take to move 64 disks?

A. 500 billion years! Bad news for the algorithm but good news to the world!

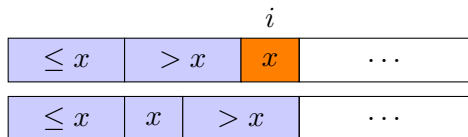
Example: Iterative Algorithm for Insertion Sort

Algorithm 3: InsertionSort(A, n)

Input: unsorted $A[n]$

Output: $A[n]$ in ascending order

```
1: for  $j \leftarrow 2$  to  $n$  do  
2:    $x \leftarrow A[j]$ ;  
3:    $i \leftarrow j - 1$  //insert  $A[j]$  to  $A[1 \dots j - 1]$ ;  
4:   while  $i > 0$  and  $x < A[i]$  do  
5:      $A[i + 1] \leftarrow A[i]$ ,  $i \leftarrow i - 1$ ;  
6:   end  
7:    $A[i + 1] \leftarrow x$ ;  
8: end
```



Worst-case Complexity

Basic computer step. element comparison

Input size. n

$$\left. \begin{array}{l} W(n) = W(n-1) + (n-1) \\ W(1) = 0 \end{array} \right\} \Rightarrow W(n) = n(n-1)/2$$

- When inserting the i -th element, algorithm compares it with the first $i-1$ sorted elements; the maximum number of comparisons is $i-1$.

Solve Recurrence Relation by Direct Iteration

$$W(n) = W(n-1) + n - 1$$

$$= (W(n-2) + n - 2) + n - 1$$

$$= W(n-2) + n - 2 + n - 1$$

$$= \dots$$

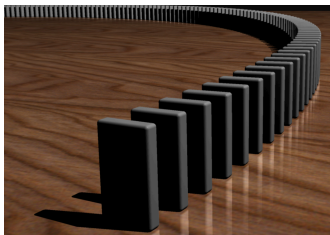
$$= W(1) + \underline{1 + 2 + \dots + (n-2) + (n-1)} // \text{reach the initial term}$$

$$= 0 + \underline{1 + 2 + \dots + (n-2) + (n-1)} // \text{substitute with initial value}$$

$$= n(n-1)/2$$

Mathematical Induction (dates back to 370 BC, Plato's Parmenides)

Mathematical induction is a mathematical proof technique $\Rightarrow$ prove that a property $P(n)$ holds for every natural number $n \in \mathbb{N}$.



Mathematical induction proves that we can climb as high as we like on a ladder, by proving that we can climb onto the bottom rung (the basis) and that from each rung we can climb up to the next one (the step). — Concrete Mathematics

Template of Mathematical Induction

The method of induction requires two facts to be proved.

Induction basis: Prove the property holds for $n = 0$.

Induction step

- 1 Prove that if the property holds for n , then it holds for $n + 1$

$$P(0) = 1$$

$$\forall n, P(n) = 1 \Rightarrow P(n + 1) = 1$$

$$n = 0, P(0) \Rightarrow P(1); n = 1, P(1) \Rightarrow P(2) \dots$$

- 2 Prove that if the property holds for all $k < n$, then it also holds for n .

$$P(0) = 1$$

$$\forall k < n, P(k) = 1 \Rightarrow P(n) = 1$$

$$n = 1, P(0) = 1 \Rightarrow P(1) = 1;$$

$$n = 2, P(0) = 1 \wedge P(1) = 1 \Rightarrow P(2) = 1 \dots$$

Comparsion Between Two Types of Mathematical Induction

Induction basis is same: $P(0) = 1$

Induction step is different

Logic reasoning:

- ① Type 1 induction: $P(0) = 1 \Rightarrow P(1) = 1 \Rightarrow P(2) = 1$
- ② Type 2 induction: $P(0) = 1 \Rightarrow P(0) = 1 \wedge P(1) = 1 \Rightarrow P(0) = 1 \wedge P(1) = 1 \wedge P(2) = 1 \Rightarrow P(0) = 1 \wedge P(1) = 1 \wedge P(2) = 1 \wedge P(3) = 1$

Think. The intuitions are the same, when to apply which?

- Type 1 (loose coupling): the property of next number only depends on its nearest preceding
- Type 2 (tight coupling): the property of next number depends on all its preceding terms

Remarks on Mathematical Induction

These two steps establish the property $P(n) = 1$ for every natural number $n = 0, 1, 2, 3$.

Remarks on Mathematical Induction

These two steps establish the property $P(n) = 1$ for every natural number $n = 0, 1, 2, 3$.

- The base case does not necessarily begin with $n = 0$. It can begin with any natural number n_0 , establishing the truth of $P(n) = 1$ holds for all $n \geq n_0$.

Remarks on Mathematical Induction

These two steps establish the property $P(n) = 1$ for every natural number $n = 0, 1, 2, 3$.

- The base case does not necessarily begin with $n = 0$. It can begin with any natural number n_0 , establishing the truth of $P(n) = 1$ holds for all $n \geq n_0$.
- The method can be extended to structural induction $\Rightarrow$ prove statements about more general well-founded structures, such as trees (widely used in mathematical logic and computer science).

Verify Correctness of Solution: Mathematical Induction

Proposition. $W(n) = n(n-1)/2$ is the general term formula for recurrence relation

$$\begin{cases} W(n) = W(n-1) + n - 1 \\ W(1) = 0 \end{cases}$$

Verify Correctness of Solution: Mathematical Induction

Proposition. $W(n) = n(n-1)/2$ is the general term formula for recurrence relation

$$\begin{cases} W(n) = W(n-1) + n - 1 \\ W(1) = 0 \end{cases}$$

Method: Mathematical Induction

- ① Basis: $n = 1$, $W(1) = 1 \times (1 - 1)/2 = 0$
- ② Induction step: $P(n) = 1 \Rightarrow P(n+1) = 1$:

$$\begin{aligned} W(n+1) &= W(n) + n \\ &= \underline{n(n-1)/2} + n \quad // \text{premise} \\ &= n((n-1)/2 + 1) = n(n+1)/2 \end{aligned}$$

Variant of Direct Iteration: Substitution-then-Iteration

When n itself is a function of another variable, say, k , and k decreases by 1 after each iteration, we first substitute n by the function of k , then apply the iteration approach over k .

- 1 Transform the recursive relation in n to recursive relation in k
- 2 Iterate over k
- 3 Transform the general term formula about k back to general term formula about n

MergeSort Algorithm

Algorithm 4: MergeSort(A, n)

Input: unsorted $A[n]$

Output: sorted $A[n]$ in ascending order

```
1:  $l \leftarrow 1, r \leftarrow n$ ;  
2: if  $l < r$  then  
3:    $k \leftarrow \lfloor (l + r)/2 \rfloor$ ;  
4:   MergeSort( $A, l, k$ );  
5:   MergeSort( $A, k + 1, r$ );  
6:   Merge( $A, p, k, r$ )  
7: end
```

Example of Substitution-and-Iteration (1/2)

Assume $n = 2^k$, the recurrence relation is:

$$\begin{cases} W(n) = 2W(n/2) + n - 1 \\ W(1) = 0 \end{cases}$$

- $n - 1$ is the cost of merge

Substitution: $n \rightarrow 2^k$

$$\begin{cases} W(2^k) = 2W(2^{k-1}) + 2^k - 1 \\ W(2^0 = 1) = 0 \end{cases}$$

Example of Substitution-and-Iteration (2/2)

$$W(n)$$

$$= 2W(2^{k-1}) + 2^k - 1 \quad // \text{substitute and iterate on } k$$

$$= 2(2W(2^{k-2}) + 2^{k-1} - 1) + 2^k - 1 \quad // \text{1st round iteration}$$

$$= 2^2W(2^{k-2}) + 2^k - 2 + 2^k - 1$$

//2nd round iteration

$$= 2^2(2W(2^{k-3}) + 2^{k-2} - 1) + 2^k - 2 + 2^k - 1$$

$$= \dots$$

$$= 2^k W(2^0 = 1) + k2^k - (2^{k-1} + 2^{k-2} + \dots + 2 + 1)$$

$$= 0 + k2^k - 2^k + 1$$

$$= n \log n - n + 1 \quad // \text{substitute back}$$

Outline

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Simplification-then-Iteration

Motivation

- Basic approach for solving recurrence relation is iteration
- When the original recurrence relation is complex, we need simplification
 - transform high order equation to one order equation (reduce dependence), then substitute

Example of QuickSort

Recap of QuickSort

Suppose the elements in $A[n]$ are distinct, set $l \leftarrow 1$, $r \leftarrow n$, partition $A[l \dots r]$ with the first element $A[l] = x$, such that

- elements less than x are stored in $A[l \dots k - 1]$
- elements greater than x are stored in $A[k + 1 \dots r]$
- $A[l]$ is placed in $A[k]$

sort $A[l \dots k - 1]$ and $A[k + 1 \dots r]$ recursively

Overall complexity.

- complexity of subproblems
- complexity of partition

Input and Subproblem Size

According to the final position of the first element x in the resulting sorted array, we can divide the inputs to n cases

final position of x	size of subproblem 1	size of subproblem 2
1	0	$n - 1$
2	1	$n - 2$
3	2	$n - 3$
$\dots$	$\dots$	$\dots$
$n - 1$	$n - 2$	1
n	$n - 1$	0

- For each input, the number of comparisons required for partition is **exactly** $n - 1$ (think why?)

Summation of Complexity

$$T(0) + T(n-1) + n - 1$$

$$T(1) + T(n-2) + n - 1$$

$$T(2) + T(n-3) + n - 1$$

$\dots$

$$T(n-1) + T(0) + n - 1$$

Summation: $2(T(1) + \dots + T(n-1)) + n(n-1)$

Average-case Complexity of QuickSort

Assumption. The first element x finally ends up in each position with equal probability:

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + O(n), n \geq 2$$

$$T(1) = 0$$

$$T(0) = 0$$

Observation and Idea

- The recurrence relation is complex: n -th term depends on all preceding terms $\leadsto$ direct iteration would be very complex
- **Idea:** Simplify the complex equation, then iterate

Simplification via Subtraction

Rewrite and iterate once to obtain two recurrence relations, then try to simplify the terms on the right-hand side.

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + n - 1$$

$$nT(n) = \boxed{2 \sum_{i=1}^{n-1} T(i)} + n(n-1)$$

$$(n-1)T(n-1) = \boxed{2 \sum_{i=1}^{n-2} T(i)} + (n-1)(n-2)$$

Simplification via Subtraction

Subtraction

$$nT(n) - \underline{(n-1)T(n-1)} = \underline{2T(n-1)} + 2(n-1)$$

Simplification

$$nT(n) = (n+1)T(n-1) + \Theta(n)$$

Rewrite

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{\Theta(n)}{\textcolor{blue}{n(n+1)}} = \frac{T(n-1)}{n} + \frac{\Theta(1)}{n+1}$$

Iteration

$$\begin{aligned}\frac{T(n)}{n+1} &= \frac{T(n-1)}{n} + \frac{\Theta(1)}{n+1} = \dots \\ &= \Theta(1) \left(\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} \right) + \frac{T(1)}{2} \text{ // reach the initial term} \\ &= \Theta(1) \left(\frac{1}{n+1} + \frac{1}{n} + \dots + \frac{1}{3} \right) \text{ // substitute with initial value} \\ &= \Theta(\ln n)\end{aligned}$$

$$T(n) = \Theta(n \log n)$$

Outline

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Concept of Recursion Tree

When $F(n)$ relies on several non-consecutive preceding terms, we can solve the recurrence relation using recursion tree.

- Recursion tree is the model of recursive computation, also the iteration of recurrence relation
- The generation of recursion tree is the same as that of recursion process
- The nodes on the recursion tree are exactly the terms in the recursive expansion
- The summation of all nodes (including the internal and leaf nodes) on the recursion tree is the solution to the recurrence relation

Representation of Iteration in Recursion Tree

Recursion tree is the model of recursion $\Rightarrow$ closely related to solving for recurrence relation

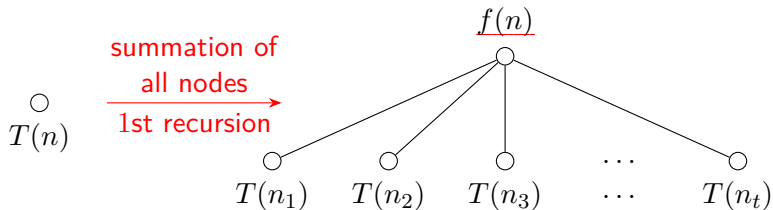
Assume the recurrence relation is as below:

$$T(n) = T(n_1) + \cdots + T(n_t) + f(n), |n_1|, \dots, |n_t| < |m|$$

- $T(n_1), \dots, T(n_t)$: function terms
- $f(n)$: dividing cost + merging cost

How do we represent $T(n)$ on the recursion tree?

Visualization of Recursion Tree

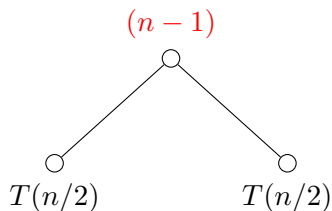


the root node represents the combine and divide cost
each leaf node is a function term

Example of 2-Level Recursion Tree

Recurrence relation for MergeSort

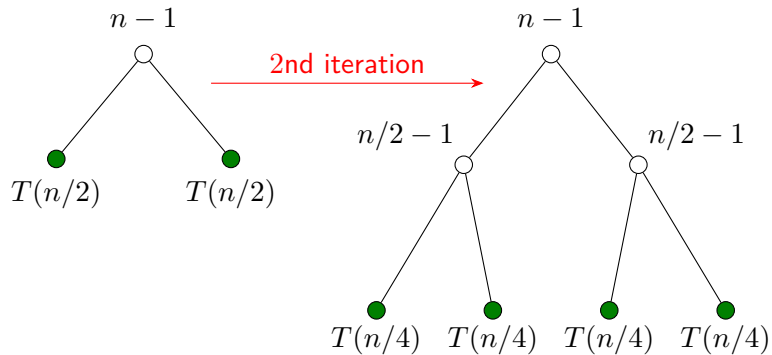
$$\begin{cases} T(n) = 2T(n/2) + (n - 1) \\ T(2) = 1 \\ T(1) = 0 \end{cases}$$



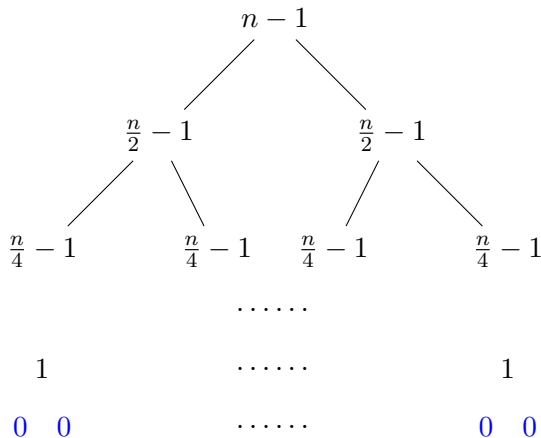
Rules for Generating a Recursion Tree

- ① At the beginning, there is only the root node in the recursion tree, whose value is $T(n)$
- ② Repeat the following steps:
 - represent the function term $T(n)$ in the leaf node by a two-level subtree
 - replace the leaf node with this subtree
- ③ Continue the generation of recursion tree until there is no function term in the tree.
 - Reaching the leaf nodes — initial values

Demo of Balanced Recursion Tree Generation



The Whole Recursion Tree



$$n - 1$$

$$n - 2$$

$$n - 4$$

$$n - 2^{k-1}$$

Calculate the Sum of Recursion Tree (balanced case)

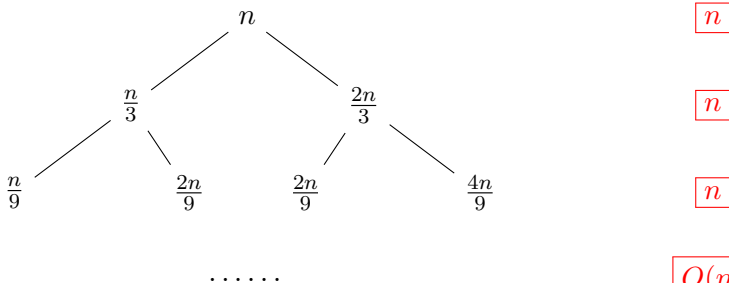
$$\begin{cases} T(n) = 2T(n/2) + n - 1, n = 2^k \\ T(1) = 0 \end{cases}$$

$$\begin{aligned} T(n) &= \overbrace{(n-1)}^{\text{0-level}} + \overbrace{(n-2)}^{\text{1-level}} + \cdots + \overbrace{(n-2^{k-1})}^{(k-1)\text{-level}} \\ &= kn - (2^k - 1) \quad // k = \log n \\ &= n \log n - n + 1 \end{aligned}$$

Application of Recursion Tree (unbalanced case)

Compute the general term formula of

$$T(n) = T(n/3) + T(2n/3) + n$$



Different routes reach the initial value at different rates

- the leftmost route is fastest – estimate the lower bound
- the rightmost route is slowest – estimate the upper bound

Calculate the Sum of Recursion Tree (unbalanced setting)

Recurrence relation: $T(n) = T(n/3) + T(2n/3) + n$

The depth of recursion tree is k , the sum at each level is $O(n)$

- Estimate the longest route to calculate for the upper bound

$$n \left(\frac{2}{3}\right)^k = 1 \Rightarrow \left(\frac{3}{2}\right)^k = n \Rightarrow k = \log_{3/2} n$$

$$T(n) < \log_{3/2} n \times n = O(n \log n)$$

- Estimate the shortest route to calculate the lower bound

$$T(n) > \log_3 n \times n = \Omega(n \log n)$$

Combining these bounds, $T(n) = \Theta(n \log n)$

Remark

For simplicity, the leaf nodes that represent initial values are omitted in the summation.

- The initial values usually cannot be represented by $f(n)$.
- The initial values are usually constants, such as 0 or 1, and thus they can be easily calculated separately.

Outline

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Application of Master Theorem

Solving recurrence relations

$$T(n) = aT(n/b) + f(n)$$

- a : the number of subproblems after dividing
 - n/b : the size of subproblems
 - $f(n)$: the cost of dividing and merging subproblems
-

Examples

- binary search: $T(n) = T(n/2) + 1$
- merge sort: $T(n) = 2T(n/2) + n - 1$

Master Theorem

Let $a \geq 1$, $b \geq 1$ be constants, $T(n)$ and $f(n)$ be functions, and

$$T(n) = aT(n/b) + f(n)$$

- ❶ if $\exists \varepsilon > 0$ s.t. $f(n) = O(n^{(\log_b a) - \varepsilon})$, then:

$$T(n) = \Theta(n^{\log_b a})$$

- ❷ if $f(n) = \Theta(n^{\log_b a})$, then:

$$T(n) = \Theta(n^{\log_b a} \log n)$$

- ❸ if $\exists \varepsilon > 0$ s.t. $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$, and $\exists r < 1$ s.t. for all n (can be relaxed to for sufficiently large n) the inequality $af(n/b) \leq rf(n)$ holds, then:

$$T(n) = \Theta(f(n))$$

How to prove the master theorem?

Direct Iteration

$$T(n) = aT(n/b) + f(n)$$

For convenience, let $n = b^k$

$$\begin{aligned} T(n) &= aT\left(\frac{n}{b}\right) + f(n) \\ &= a\left(aT\left(\frac{n}{b^2}\right) + f\left(\frac{n}{b}\right)\right) + f(n) \\ &= a^2T\left(\frac{n}{b^2}\right) + af\left(\frac{n}{b}\right) + f(n) \\ &= \dots \end{aligned}$$

Result of Iteration

$$= a^k T\left(\frac{n}{b^k}\right) + a^{k-1} f\left(\frac{n}{b^{k-1}}\right) + \cdots + a f\left(\frac{n}{b}\right) + f(n)$$

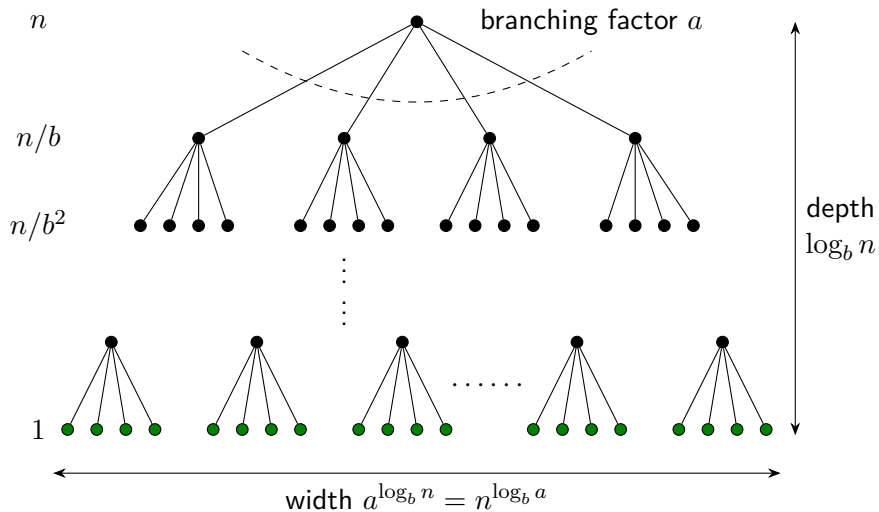
$$= a^k T(1) + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \quad // \text{reach the initial term}$$

$$= \underline{c_1 n^{\log_b a}} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \quad // \text{assume } T(1) = c_1$$

$$k = \log_b n = \log_b a \cdot \log_a n$$

- the first term is the total cost of all **base** subproblems
- the second term is the total cost of all dividing and merging steps

Corresponding Recursion Tree



Explaining the Meaning of Recursion Tree

- a : branching factor
- b : the size of subproblems is reduced by a factor of b with each level of recursion
- $k = \log_b n$: reaches the base case after k levels, the height of the recursion tree

the j th level of the tree consists of a^j subproblems, each of size n/b^j

Case 1

$$\exists \varepsilon > 0 \text{ s.t. } f(n) = O(n^{(\log_b a) - \varepsilon})$$

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{k-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + O\left(\sum_{j=0}^{(\log_b n)-1} a^j \left(\frac{n}{b^j}\right)^{(\log_b a) - \varepsilon}\right) \text{ // substitute using the assumption} \\ &= c_1 n^{\log_b a} + O\left(n^{(\log_b a) - \varepsilon} \sum_{j=0}^{(\log_b n)-1} \frac{a^j}{(b^{(\log_b a) - \varepsilon})^j}\right) \\ &\text{ // move sum irrelevant terms outside} \end{aligned}$$

Case 1: Continue to Simplify

$$\boxed{\frac{1}{(b^{\log_b a - \varepsilon})^j} = \frac{b^{\varepsilon j}}{(b^{\log_b a})^j} = \frac{b^{\varepsilon j}}{a^j}}$$

$$= c_1 n^{\log_b a} + O \left(n^{(\log_b a) - \varepsilon} \sum_{j=0}^{\log_b n - 1} \frac{a^j}{(b^{(\log_b a) - \varepsilon})^j} \right) \quad // \text{simplify}$$

$$= c_1 n^{\log_b a} + O \left(n^{(\log_b a) - \varepsilon} \boxed{\sum_{j=0}^{\log_b n - 1} (b^{\varepsilon})^j} \right) \quad // \text{geometric series}$$

$$= c_1 n^{\log_b a} + O \left(n^{(\log_b a) - \varepsilon} \frac{b^{\varepsilon \log_b n} - 1}{b^{\varepsilon} - 1} \right) \quad // \text{ignore the constants}$$

$$= c_1 n^{\log_b a} + O \left(n^{(\log_b a) - \varepsilon} n^{\varepsilon} \right) = \Theta(n^{\log_b a})$$

Case 2

$$f(n) = \Theta(n^{\log_b a})$$

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{\log_b n - 1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + \Theta\left(\sum_{j=0}^{(\log_b n) - 1} a^j \boxed{\left(\frac{n}{b^j}\right)^{\log_b a}}\right) \quad // \text{substitute with premise} \end{aligned}$$

// move sum irrelevant terms outside

$$\begin{aligned} &= c_1 n^{\log_b a} + \Theta\left(n^{\log_b a} \sum_{j=0}^{(\log_b n) - 1} \frac{a^j}{a^j}\right) \\ &= c_1 n^{\log_b a} + \Theta(n^{\log_b a} \log_b n) = \Theta(n^{\log_b a} \log n) \end{aligned}$$

Case 3

$$\exists \varepsilon > 0, f(n) = \Omega(n^{(\log_b a) + \varepsilon}) \quad (1)$$

$$af(n/b) \leq rf(n) \quad (2)$$

Repeatedly apply Condition (2)

$$a^j f\left(\frac{n}{b^j}\right) \leq a^{j-1} r f\left(\frac{n}{b^{j-1}}\right) \leq \dots \leq r^j f(n)$$

$$\begin{aligned} T(n) &= c_1 n^{\log_b a} + \sum_{j=0}^{(\log_b n)-1} a^j f\left(\frac{n}{b^j}\right) \\ &= c_1 n^{\log_b a} + f(n) + \sum_{j=1}^{(\log_b n)-1} a^j f\left(\frac{n}{b^j}\right) \end{aligned}$$

Case 3 (continue)

$$\begin{aligned} T(n) &\leq c_1 n^{\log_b a} + f(n) + \sum_{j=1}^{(\log_b n)-2} \boxed{r^j f(n)} \\ &= c_1 n^{\log_b a} + f(n) + f(n) r \frac{1 - r^{(\log_b n)-2}}{1 - r} // \text{geometric series: } r < 1 \\ &= c_1 n^{\log_b a} + \Theta(f(n)) \end{aligned}$$

$$\text{Condition 1} \Rightarrow \text{order}(f(n)) \geq \text{order}(n^{\log_b a})$$

Therefore, we have:

$$T(n) = \Theta(f(n))$$

- **Recap:** Condition 2 is used to prove the coefficient of $f(n)$ is upper bounded by a constant.

Simplified Form of Master Theorem



Define $h(n) = n^{\log_b a}$, we re-state master theorem as below:

$$T(n) = \begin{cases} \Theta(h(n)) & \text{if } f(n) = o(h(n)) \\ \Theta(h(n) \log n) & \text{if } f(n) = \Theta(h(n)) \\ \Theta(f(n)) & \text{if } f(n) = \omega(h(n)) \wedge \exists r < 1 \text{ s.t. } af(n/b) < rf(n) \end{cases}$$

Outline

- 1 Sequences and Series Summation
- 2 Recurrence Relation and Algorithm Analysis
 - Approach 1: Direct Iteration
 - Approach 2: Simplification-then-Iteration
 - Approach 3: Recursion Tree
- 3 Master Theorem and Its Proof
- 4 Application of Master Theorem

Example 1: Solving a Recurrence Relation

Compute the general term formula of recurrence relation:

$$T(n) = 9T(n/3) + n$$

Applying the master theorem

- $a = 9, b = 3, h(n) = n^2, \varepsilon = 1;$
- $f(n) = n, n^{\log_3 9} = n^2, f(n) = O(n^{\log_3 9 - 1}) = o(n^2)$

Master theorem (case 1) $\Rightarrow T(n) = \Theta(n^2)$

Example 2: Solving a Recurrence Relation

Compute the general term formula of recurrence relation:

$$T(n) = T(2n/3) + 1$$

Applying the Master theorem

- $a = 1, b = 3/2, h(n) = n^{\log_b a} = n^0 = 1;$
- $f(n) = 1, n^{\log_{3/2} 1} = n^0 = 1, f(n) = \Theta(1)$

Master theorem (case 2) $\Rightarrow T(n) = \Theta(\log n)$

Example 3 of Solving Recurrence Relation

Compute the general term formula of recurrence relation:

$$T(n) = 3T(n/4) + n \log n$$

Applying the Master theorem

- $a = 3, b = 4, h(n) = n^{\log_4 3}$;
- $f(n) = n \log n = \Omega(n^{\log_4 3 + \varepsilon}) \approx \Omega(n^{0.793 + \varepsilon})$, choose $\varepsilon = 0.2$
- Check addition condition $af(n/b) \leq rf(n)$ holds for all n .
 - Test $f(n) = n \log n \Rightarrow af(n/b) = 3(n/4) \log(n/4) \leq rn \log n$ holds for some $r < 1$.
 - Choose $r = 3/4 < 1$, this inequality holds for all n .

Master theorem (case 3) $\Rightarrow T(n) = \Theta(f(n)) = \Theta(n \log n)$

Complexity Analysis of Recursive Algorithms

Binary search. $T(n) = T(n/2) + 1$, $T(1) = 1$

- $a = 1$, $b = 2$, $h(n) = n^{\log_2 1} = 1$, $f(n) = 1$

Master theorem (case 2) $\Rightarrow T(n) = \Theta(\log n)$

Merge sort. $T(n) = 2T(n/2) + (n - 1)$, $T(1) = 0$

- $a = 2$, $b = 2$, $h(n) = n^{\log_2 2} = n$, $f(n) = n - 1$

Master theorem (case 2) $\Rightarrow T(n) = \Theta(n \log n)$

Cases that Master Theorem is not Applicable

Example. Compute the general term formula of

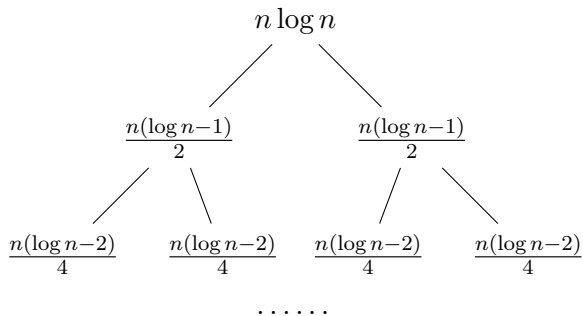
$$T(n) = 2T(n/2) + n \log n$$

Applying the Master theorem. $a = b = 2$, $h(n) = n^{\log_b a} = n$, $f(n) = n \log n$

Only case 3 is possible, but $\nexists r < 1$ to make $af(n/b) \leq rf(n)$ hold for all n .

$$\begin{aligned} af(n/b) - rf(n) &= 2(n/2) \log(n/2) - rn \log n \\ &= n(\log n - 1) - rn \log n \\ &= (1 - r)n \log n - n > 0 \text{ if } r < 1 \end{aligned}$$

Solving via Recursion Tree



$$n \log n$$

$$n(\log n - 1)$$

$$n(\log n - 2)$$

$$n(\log n - k + 1)$$

Summation

$$\begin{aligned}T(n) &= n \log n + n(\log n - 1) + n(\log n - 2) \\&\quad + \cdots + n(\log n - k + 1) \\&= (n \log n) \log n - n(1 + 2 + \cdots + k - 1) \\&= n \log^2 n - nk(k - 1)/2 \quad // \text{substitute with } k = \log n \\&= \Theta(n \log^2 n)\end{aligned}$$

Summary

Classical sequences and series

Complexity analysis: solving recurrence relation

- Direct Iteration
- Simplification-then-Iteration
- Recursion Tree

Master theorem and its proof

Application of Master Theorem